Dating sim Checkpoint 2

Xavier Bradley 1730298 , Alexandra Wintle 1731754

Champlain College

420-N44-LA GAME DEVELOPMENT

Brendan Wood

April 7th 2026

Final Project Description

Our final project is a narrative-driven dating simulation game that combines two different gameplay styles into one experience. The first style is a retro-inspired 2D pixel exploration system where the player can move around a small house environment using keyboard controls such as WASD or arrow keys. The second style is a traditional dating simulator interface, where the player interacts with characters through dialogue scenes that include character portraits, backgrounds, and multiple-choice responses. The game takes place entirely inside a house during a storm, where the player is stuck with a small group of characters. When the player explores the environment, they can approach and interact with different characters. This interaction triggers a transition from the pixel exploration mode into a dialogue-based scene. During these scenes, the player is presented with multiple dialogue options (such as A, B, C, or D choices), and each decision directly affects relationship levels with the characters. A key mechanic of the game is the relationship system, where the player's choices influence how much each character likes or dislikes them. These relationship levels determine which events, dialogue paths, and ultimately which ending the player will receive. The game will include multiple endings based on these interactions, encouraging replayability.

The game features one main playable character and two potential love interests. The experience is designed to be inclusive, with a focus on LGBTQ+ relationships, while still allowing flexibility in character design depending on development progress. The game world is intentionally small and manageable, consisting of a main open-concept area (living room, kitchen, and dining space) along with a few additional rooms such as a bedroom, bathroom, or garage. This keeps the scope realistic while still allowing for meaningful interactions.

A full playthrough of the game is expected to last approximately 15 to 20 minutes, focusing heavily on dialogue, character interaction, and narrative progression rather than complex mechanics or combat systems.

Why We Chose This Project

We chose this project because we wanted to create something unique and different from the typical game ideas in the class. While many projects focus on action or gameplay-heavy mechanics, our game focuses on storytelling, character relationships and player choice.

Another reason we chose this idea is because it allows us to combine creativity with manageable technical complexity. By limiting the environment to a house and focusing on dialogue systems, we can

create a polished and engaging experience without making the project too large or difficult to complete within the given timeframe.
Combining it with a pixel exploration system makes the game feel more interactive and dynamic, similar to how games like Pokémon switch between exploration and interaction modes.
Overall, we believe this project is both achievable and engaging, and it allows us to build a game that is story-driven, interactive, and memorable.

Synopsis of the Game

The game is a narrative-driven dating simulation where the player is trapped inside a house during a storm with a small group of characters. The main goal of the player is to interact with these characters and build relationships in order to end up with the love interest that appeals to them the most. As the player progresses, they will navigate conversations, make choices, and influence how each character feels about them. Although the storm serves as the reason everyone is stuck together, the main tension comes from the relationships between characters, including potential jealousy and competition as feelings begin to develop. The game blends romantic storytelling with light humor and chaotic interactions, creating an engaging and replayable experience where each playthrough can lead to a different outcome.

Story of the Game

The game begins with a casual “boys night” at the player’s house, where the main character has invited two important people: a childhood friend and a newer college friend. The atmosphere starts off relaxed and friendly, with light conversation and interactions between the characters. Early on, the player is introduced to both characters and begins forming initial impressions through dialogue choices. Suddenly, the tone shifts when all characters receive an emergency alert on their phones warning them to stay inside due to dangerous weather conditions. This alert confirms that a severe storm has made it unsafe to leave, forcing everyone to remain inside the house. From this point forward, the characters are confined together, creating the foundation for both bonding and conflict. As the story progresses, various events begin to unfold. One major event includes a power outage caused by the storm, which the player must resolve by exploring the house and interacting with the environment. During this time, tensions begin to rise between the characters. An argument eventually breaks out, likely driven by jealousy or emotional conflict as relationships begin to develop. These moments create opportunities for the player to make meaningful decisions that impact how each character perceives them.

Throughout the game, the player's choices play a critical role in shaping the story. Dialogue decisions directly affect relationship levels, and the amount of time spent interacting with each character influences which story paths are unlocked. The player must choose who to prioritize, knowing that these decisions will have lasting consequences on the narrative.The game features three possible endings. The player can end up forming a romantic relationship with the childhood friend, pursue a relationship with the newer college friend, or fail to build a strong enough connection and end up alone. Each ending reflects the cumulative impact of the player's choices, encouraging replayability and exploration of different story paths.

Rules of the Game

The game is governed by a set of rules that define how the player can interact with the environment and characters. The player is free to explore the house and move between rooms at any time, as long as they are not actively engaged in a dialogue scene. Movement allows the player to approach characters and trigger interactions that advance the story. The player can choose to speak with any character, but interactions are influenced by ongoing events within the game. During certain situations, such as a power outage, the player must complete specific tasks before progressing to more meaningful conversations. Without completing these events, dialogue may be limited or repetitive, encouraging the player to actively engage with the game's objectives. The game also includes an inventory system, where the player can collect and use items found within the house. These items are necessary for completing certain tasks, such as fixing the power, and can influence how events unfold.

Character relationships are central to the game's rules. Each interaction affects relationship levels, and consistently ignoring a character will cause their interest in the player to decrease over time. This directly impacts which story paths and endings become available. The only failure condition in the game occurs when the player does not build a strong enough relationship with any character, resulting in an ending where the player is left alone. There are no traditional "game over" states during gameplay; instead, outcomes are determined by the player's choices and level of engagement throughout the story.

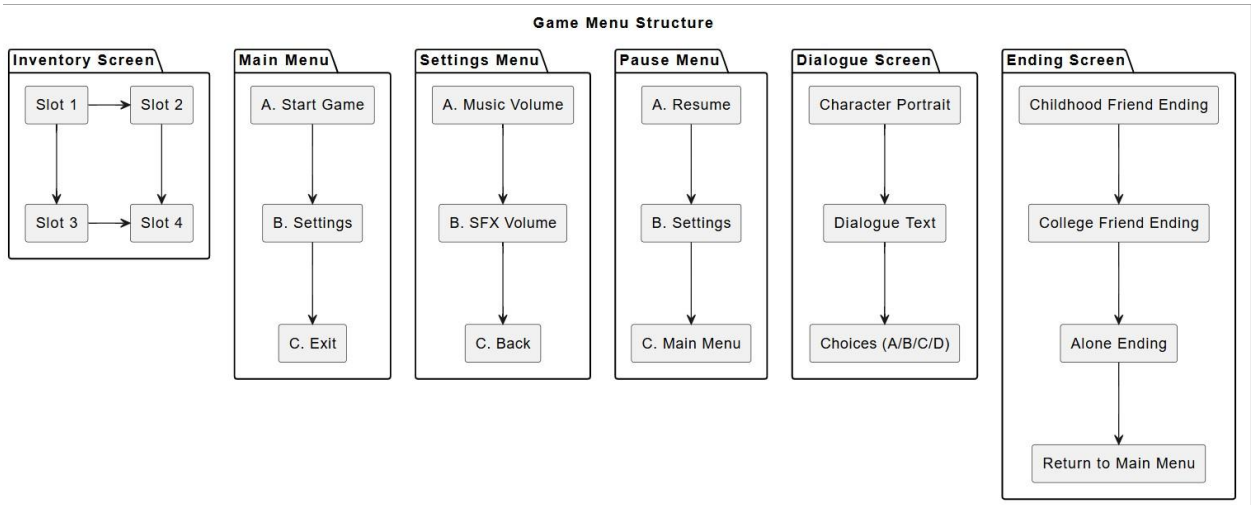
Mechanics of the Game (Keyboard Controls)

The game mechanics are centered around player movement, interaction and dialogue selection. The player navigates the environment using either the WASD keys or the arrow keys to move freely throughout the house. To interact with characters or objects in the environment, the player presses the **E key** when positioned near them. This action triggers dialogue scenes or event interactions. During dialogue sequences, the player selects responses by clicking on their chosen option using the left mouse button (M1). These choices directly impact relationship levels and story progression. The game also includes an inventory system, which can be accessed by pressing the **I key**. This allows the player to view and use collected items that may be required for completing certain tasks or events.

Finally, the player can access the pause or menu screen at any time by pressing the **Escape (ESC) key**, allowing them to pause the game or navigate additional options.

3.

<u>Overall</u>



<u>Main Menus Detailed:</u>

Main Menu Screen

GAME TITLE

A. Start Game
B. Settings
C. Exit

Settings Menu Screen

SETTINGS

A. Music Volume
B. SFX Volume
C. Back

Pause Menu Screen

PAUSED

A. Resume
B. Settings
C. Main Menu

Inventory Screen

INVENTORY

[Slot 1] [Slot 2]

[Slot 3] [Slot 4]

Dialogue Screen

BACKGROUND IMAGE

CHARACTER PORTRAIT

DIALOGUE TEXT BOX

A. Response Option 1
B. Response Option 2
C. Response Option 3
D. Response Option 4

Gameplay Screen

HOUSE / ROOM AREA

PLAYER CHARACTER
NPC CHARACTER

Press E to Interact

HUD: I = Inventory ESC = Pause

Ending Screen

ENDING TITLE

Final Dialogue / Ending Description

A. Return to Main Menu

Explanation of Menu Items and How They Are Handled by Code

Each menu item in the game is connected to specific actions in the code that control navigation between different screens and core gameplay functions. These actions are handled through button inputs and key presses, which trigger changes in the game's state or user interface.

In the main menu selecting "Start Game" begins the game by loading the gameplay scene using a scene management function. Choosing "Settings" switches the interface to the settings menu, while selecting "Exit" closes the application using a built-in quit command.

In the settings menu, the player can adjust both music and sound effects volume. These options are connected to the game's audio system, which updates volume levels in real time. The "Back" option returns the player to the previous screen by switching the active user interface. (we have yet to add audio)

During gameplay, player movement is handled through keyboard input using the WASD or arrow keys. When the player presses the E key near a character, the game checks for interaction range and transitions into the dialogue screen. Pressing the ESC key pauses the game and opens the pause menu, while pressing the I key toggles the inventory screen on and off.

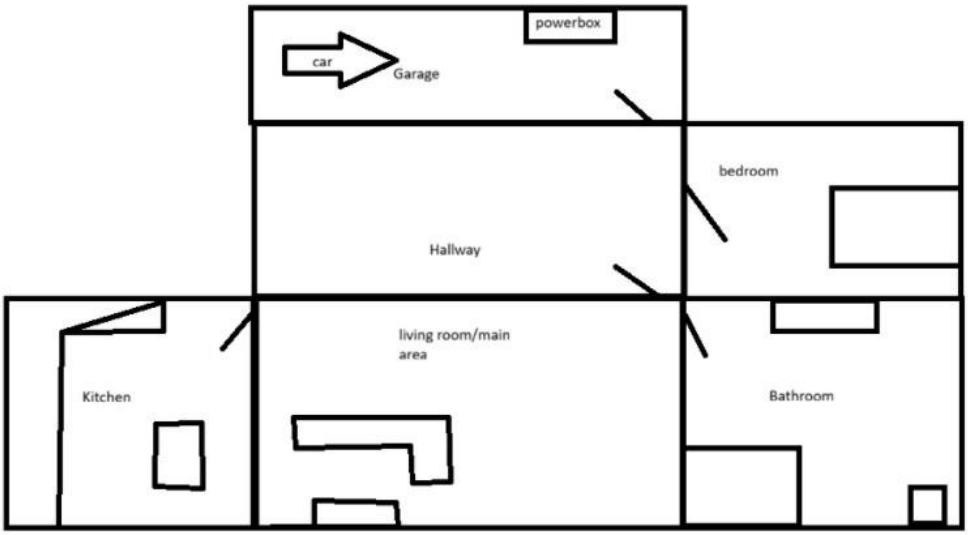
The pause menu allows the player to resume the game, access the settings menu, or return to the main menu. Resuming simply restores gameplay, while returning to the main menu reloads the starting scene and ends the current session.

The inventory system is displayed as a simple interface with four slots. When opened, the code updates the user interface to show any collected items and allows the player to view them for use in events.

The dialogue screen presents character interactions using selectable response options. When the player clicks a response, the code updates relationship values and determines the next part of the conversation. These choices directly influence how the story progresses.

Finally, the ending screen is displayed once the story is complete. The game determines which ending to show based on the player's relationship levels with each character. From this screen, the player can return to the main menu, allowing them to restart the game and explore different outcomes.

Gameplay Layout:



TimeLine:

Project Timeline

Week	Task	Task Owner	Dependencies / Blockers	Time (Hours)	Priority	Status
12	Finalize game concept and story	AW, XB	Group agreement on final idea	3	1	Completed
12	Design house layout and events	AW	Final concept decided	3	1	Completed
12	Create design diagrams (UML, menu structure, wireframes, and layout diagrams)	AW	Final concept decided	4	1	Completed
12	Create menu UI (Main Menu, Settings)	XB	Unity setup	4	1	In Progress
12	Implement player movement (WASD)	XB	Unity scene ready	3	1	In Progress
13	Create character designs (main character and love interests)	AW	Final story direction	3	1	Not Started
13	Implement character sprites into the game	AW	Character designs complete	3	1	Not Started
13	Build house map in Unity (main structure of the home)	AW	House blueprint complete	4	1	Not Started
13	Add furniture and environment details to the map	AW	Base house map complete	3	2	Not Started
13	Implement interaction system (E key)	XB	Player movement complete	3	1	Not Started
13	Create dialogue system (basic UI + choices)	XB	UI structure complete	4	1	Not Started

13	Finalize dialogue script and story writing	AW	Initial story drafted	3	1	Not Started
Week	Task	Task Owner	Dependencies / Blockers	Time (Hours)	Priority	Status
14	Implement relationship system	AW	Dialogue system working	4	1	Not Started
14	Add events (power outage, argument scene)	AW, XB	Dialogue + interaction system	4	1	Not Started
14	Create inventory system (4 slots)	AW	UI + interaction system	3	2	Not Started
15	Connect choices to multiple endings	AW	Relationship system complete	4	1	Not Started
15	Implement ending screens	XB, AW	Ending logic complete	3	1	Not Started
15	Polish UI and fix bugs	AW, XB	All systems implemented	4	1	Not Started
16	Final testing and debugging	AW, XB	Full game complete	4	1	Not Started
16	Record demo video	AW, XB	Game stable	2	1	Not Started
16	Prepare final submission and presentation	AW, XB	All work complete	2	1	Not Started

Code snippets:

Walking:

```
using UnityEngine;

Unity Script (2 asset references) | 0 references
public class BetaTestWalkFiller : MonoBehaviour
{
    public float moveSpeed = 5f;

    private Rigidbody2D rb;
    private Vector2 movement;

    Unity Message | 0 references
    void Start()
    {
        rb = GetComponent<Rigidbody2D>();
    }

    Unity Message | 0 references
    void Update()
    {

        movement.x = Input.GetAxisRaw("Horizontal");
        movement.y = Input.GetAxisRaw("Vertical");
    }

    Unity Message | 0 references
    void FixedUpdate()
    {
        if (movement != null) {
        rb.linearVelocity = movement.normalized * moveSpeed;
    }
}
```

Dialogue overlay:

```
using UnityEngine;

Unity Script (2 asset references) | 0 references
public class Dialogue : MonoBehaviour
{
    public GameObject dialogueUI;
    public GameObject livingRoomText;
    private bool playerInRange = false;

    Unity Message | 0 references
    void Update()
    {
        if (playerInRange && Input.GetKeyDown(KeyCode.E))
        {
            dialogueUI.SetActive(true);
            livingRoomText.SetActive(false);
        }
    }

    Unity Message | 0 references
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            playerInRange = true;
        }
    }

    Unity Message | 0 references
    private void OnTriggerExit2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            playerInRange = false;
            dialogueUI.SetActive(false);
            livingRoomText.SetActive(true);
        }
    }
}
```

Main menu code:

```
using UnityEditor;
using UnityEngine;
using UnityEngine.SceneManagement;

Unity Script (1 asset reference) | 0 references
public class MainMenuButtons : MonoBehaviour
{
    // Start is called once before the first execution of Update after the MonoBehaviour is created
    Unity Message | 0 references
    void Start()
    {

    }

    // Update is called once per frame
    Unity Message | 0 references
    void Update()
    {

    }

    0 references
    public void StartGame()
    {
        SceneManager.LoadScene("LivingRoom");
    }

    0 references
    public void Quit()
    {
        Debug.Log("Quitting game...");
    #if UNITY_EDITOR
        EditorApplication.isPlaying = false;
    #else
    Application.Quit();
    #endif
    }
}
```

Pause menu overlay:

```
3   public class PauseManager : MonoBehaviour
4   {
5       public static PauseManager Instance;
6
7       public GameObject pauseMenuUI;
8       private bool isPaused = false;
9
        Unity Message | 0 references
10      void Awake()
11      {
12          // Singleton pattern (only one instance)
13          if (Instance == null)
14          {
15              Instance = this;
16              DontDestroyOnLoad(gameObject);
17          }
18          else
19          {
20              Destroy(gameObject);
21          }
22      }
23
        Unity Message | 0 references
24      void Update()
25      {
26          if (Input.GetKeyDown(KeyCode.Escape))
27          {
28              if (isPaused)
29                  Resume();
30              else
31                  Pause();
32          }
33      }
34
```

```
1 reference
public void Pause()
{
    pauseMenuUI.SetActive(true);
    Time.timeScale = 0f;
    isPaused = true;
}

1 reference
public void Resume()
{
    pauseMenuUI.SetActive(false);
    Time.timeScale = 1f;
    isPaused = false;
}
}
```

Room change code:

```
using UnityEngine;
using UnityEngine.SceneManagement;

Unity Script (4 asset references) | 0 references
public class ChangeRooms : MonoBehaviour
{
    public string sceneToLoad;

    Unity Message | 0 references
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            SceneManager.LoadScene(sceneToLoad);
        }
    }

}
```